

Understanding Query Cost

To optimize a query, we first need to understand how to calculate its cost.



A "cost" represents the actual system resources—specifically **CPU time, memory, and Disk I/Os**—required to execute a request.



The database optimizer's primary goal is to find the access plan with the **absolute lowest total cost**.

Pages

- Depending on the storage model (**row store vs. column store**), rows are stored and read in logical **pages**.
- Each page has a fixed size (e.g., **8KB in PostgreSQL, 16KB in MySQL**).
- Assume each page holds 3 rows: $100 \text{ rows} \div 3 \approx$ **34 pages**.

row_id	emp_id	emp_name	emp_dob	emp_salary
1	101	Omar	6/14/1991	\$85,000
2	202	Youssef	11/8/1985	\$145,000
3	303	Karim	2/21/1994	\$190,000
...	...	...	...	...
100	101000	Daniel	9/5/2001	\$275,000

Page 0

1, 101, Omar,
6/14/1991, \$85,000
2, 202, Youssef,
11/8/1985, \$145,000
3, 303, Karim,
2/21/1994, \$190,000

Page 1

(Rows 4, 5, 6) ...

Page 2

(Rows 7, 8, 9) ...

...

Page 34

More rows ...
1000, 101000, Daniel,
9/5/2001, \$275,000

I/O

- A **disk I/O** (input/output) operation is a request to move data between disk and memory. For database systems, the number of disk I/Os required by a query is an important measure of its cost.
- A database reads data in pages rather than individual rows. A page read brings all of the rows stored in that page into memory.
- Because an I/O brings an entire page into memory, the other rows on that page are available without requiring additional page reads.
- Reducing the number of pages a query needs to access generally reduces its I/O cost, which is why database systems try to minimize unnecessary page reads.
- Not every page read requires a physical disk access. If the required page is already in the operating system's cache, the database can retrieve it from memory instead.
- Cached reads are still not free. The database may still need to process, interpret, and materialize the page's contents before the query can use the rows.

Page 0

1, 101, Omar,
6/14/1991, \$85,000
2, 202, Youssef,
11/8/1985, \$145,000
3, 303, Karim,
2/21/1994, \$190,000

Page 1

(Rows 4, 5, 6) ...

Page 2

(Rows 7, 8, 9) ...

...

Page 34

More rows ...
1000, 101000, Daniel,
9/5/2001, \$275,000

The Heap

- The **heap** is the data structure that stores the table's pages one after another.
- It holds the actual data: every row and every column.
- Traversing the heap is **expensive**: finding a few rows means reading a lot of data.
- That is why we need **indexes**: they tell us exactly which page(s) of the heap to pull.

Heap

Page 0

1, 101, Omar,
6/14/1991, \$85,000
2, 202, Youssef,
11/8/1985, \$145,000
3, 303, Karim,
2/21/1994, \$190,000

Page 1

(Rows 4, 5, 6) ...

Page 2

(Rows 7, 8, 9) ...

...

Page 34

More rows ...
1000, 101000, Daniel,
9/5/2001, \$275,000

Example: The Data File

- Data file: **EMPLOYEE**(NAME, SSN, ADDRESS, JOB, SAL, ...)
- Suppose that:
 - Record size **R = 150 bytes**
 - Block size **B = 512 bytes**
 - Number of records **r = 30,000**
- Then we get:
 - Blocking factor $Bfr = B \div R = 512 \div 150 = 3 \text{ records/block}$
 - File blocks $b = r / Bfr = 30,000 / 3 = 10,000 \text{ blocks}$
- Theoretical cost of a full scan to find one record:
 - Worst case: **10,000 disk I/Os** (read every block)
 - Average case: **5,000 disk I/Os** ($b / 2 = 10,000 / 2$)

Data file

Block 0

Record 1
Record 2
Record 3
450 of 512 bytes used

Block 1

Records 4–6

Block 2

Records 7–9

...

Block 9,999

Records 29,998–30,000

Example: The SSN Index

SSN index

- For an index on the **SSN** field, assume:
 - SSN field size $V_{SSN} = \mathbf{9 \text{ bytes}}$
 - Record pointer size $P_R = \mathbf{7 \text{ bytes}}$
- Then we get:
 - Index entry size $R_i = V_{SSN} + P_R = 9 + 7 = \mathbf{16 \text{ bytes}}$
 - Blocking factor $Bfr_i = B \text{ div } R_i = 512 \text{ div } 16 = \mathbf{32 \text{ entries/block}}$
 - Index blocks $b_i = r / Bfr_i = 30,000 / 32 \approx \mathbf{938 \text{ blocks}}$
 - Binary search on the index: $\log_2 938 \approx \mathbf{10 \text{ block accesses}}$

Index block 0

32 entries $\times$ 16 bytes
SSN (9 B) + pointer (7 B)
→ points into data blocks

Index block 1

Entries 33–64

Index block 2

Entries 65–96

...

Index block 937

Last entries, up to record
30,000

Comparing Access Cost

- Linear search on the unordered file averages $b/2 = 10,000 / 2 = \mathbf{5,000}$ **block accesses**.
- If the file were ordered by SSN, binary search would need $\log_2 10,000 \approx \mathbf{14}$ **block accesses**.
- Binary search on the SSN index needs $\log_2 938 \approx \mathbf{10}$ **block accesses**.
- The index cuts the cost **500×** compared with a linear scan.

5,000 block accesses
Linear search · unordered file

14 block accesses
Binary search · ordered file

10 block accesses
Binary search · SSN index